

Lab 4 - Hazards

All Project Files:

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:  J. Callenes
//
// Create Date: 01/04/2019 04:32:12 PM
// Design Name:
// Module Name: PIPELINED_OTTER_CPU
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////

module OTTER_MCU(input CLK,
                input INTR,
                input RESET,
                input [31:0] IOBUS_IN,
                output [31:0] IOBUS_OUT,
                output [31:0] IOBUS_ADDR,
                output logic IOBUS_WR
);

//Signals-----
logic [31:0] instr, pc, pc_4;
logic [31:0] rs1, rs2, regdata;
logic[31:0] U_Type, I_Type, S_Type, B_Type, J_Type;
logic [31:0] ImmExtend, alu_a_mux, alu_b_mux, pcMux, alu_res;
logic [31:0] jal, branch, jalr;
logic [3:0] alu_ctrl;
logic [2:0] pc_ctrl;
logic [1:0] rf_wr_ctrl, alu_a_ctrl, alu_b_ctrl;
logic mem_we, rf_we;
logic [31:0] mem_data;
logic[4:0] rf_wa;
```

```

//Hazard Signals
logic [1:0] ForwardMuxA,ForwardMuxB;
logic [31:0] HazardMuxB_out,HazardMuxA_out;
logic StallF,StallD,FlushE,FlushD;

//Branch signals
logic is_branchE, is_jalrE, is_jalE;
logic branch_cond, branch_takenE;
logic beq,blt,bltu,branch_Happen;
wire [2:0] actual_pc_selE;

//-----

// IF ? ID only needs the fetched instruction and PCs
typedef struct packed {
    logic [31:0] instr;
    logic [31:0] pc;
    logic [31:0] pc_4;
} if_id_t;

// ID ? EX holds decoded operands, immediates, and control signals
typedef struct packed {
    logic [31:0] instr;
    // from register file
    logic [31:0] rs1, rs2;
    // immediates
    logic [31:0] U_Type, I_Type, S_Type, B_Type, J_Type;
    // control signals
    logic rf_we;
    logic [1:0] rf_wr_ctrl;
    logic [4:0] rf_wa;
    logic mem_we;
    logic [1:0] alu_a_ctrl, alu_b_ctrl;
    logic [3:0] alu_ctrl;
    logic [2:0] pc_ctrl;
    // forwarded PCs
    logic branch_Happen;
    logic jal_Happen;
    logic jalr_Happen;
    logic [31:0] pc, pc_4;
    logic [4:0] RS1, RS2;
} id_ex_t;

// EX ? MEM holds the ALU result, data-to-store, and control
typedef struct packed {
    logic [31:0] instr;
    logic [31:0] alu_res;
    logic [31:0] rs2; // for store

```

```

    logic      mem_we;
    logic      rf_we;
    logic [1:0] rf_wr_ctrl;
    logic [4:0] rf_wa;
    logic [31:0] pc_4;      // for JAL/JALR writeback
} ex_mem_t;

// MEM ? WB holds memory output or ALU result plus control
typedef struct packed {
    logic [31:0] instr;
    logic [31:0] mem_data;
    logic [31:0] alu_res;
    logic      rf_we;
    logic [1:0] rf_wr_ctrl;
    logic [4:0] rf_wa;
    logic [31:0] pc_4;
} mem_wb_t;

if_id_t  IF_ID;
id_ex_t  ID_EX;
ex_mem_t EX_MEM;
mem_wb_t MEM_WB;

//==== Fetch =====
    logic pcWrite = 1'b1; //Hardwired high, assuming no hazards

//Mux for PC
    four_input_Mux #(.n(32)) PCMUX (
        .SEL    (actual_pc_selE),
        .D0     (pc_4),
        .D1     (jalr),
        .D2     (branch),
        .D3     (jal),
        .D_OUT  (pcMux) );

// The PC_COUNTER
    PC_Counter #(.n(32)) PC_Counter (
        .data(pcMux),
        .ld (!StallF),
        .clk(CLK),
        .clr(RESET),
        .pc(pc));

assign pc_4 = pc + 4;

I_MEM InstructionMem(// reading from instruction mem
    .imem_a(pc),
    .imem_out(instr));

```

```

// IF_ID register
always_ff @(posedge CLK) begin
if (RESET || FlushD) begin
    IF_ID <= 0;
end else if (!StallD)begin
    IF_ID.instr    <= instr;
    IF_ID.pc       <= pc;
    IF_ID.pc_4     <= pc_4;
end
end

//==== Instruction Decode
=====

//REG_FILE Decode , its mux is in the write back stage
    RegFile RegFile (
        .w_data (regdata),
        .clk     (CLK),
        .en      (MEM_WB.rf_we),
        .adr1    (IF_ID.instr[19:15]),
        .adr2    (IF_ID.instr[24:20]),
        .w_adr   (MEM_WB.rf_wa), // this needs to be RD from writeback
        .rs1     (rs1),
        .rs2     (rs2) );

assign rf_wa = IF_ID.instr[11:7];

//IMMED GEN assignments
assign U_Type = {IF_ID.instr[31:12],{12{1'b0}}};
assign I_Type = {{21{IF_ID.instr[31]}},IF_ID.instr[30:25],IF_ID.instr[24:20]};
assign S_Type = {{21{IF_ID.instr[31]}},IF_ID.instr[30:25],IF_ID.instr[11:7]};
assign B_Type =
{{20{IF_ID.instr[31]}},IF_ID.instr[7],IF_ID.instr[30:25],IF_ID.instr[11:8],1'b0};
assign J_Type =
{{12{IF_ID.instr[31]}},IF_ID.instr[19:12],IF_ID.instr[20],IF_ID.instr[30:21],1'b0};

// The decoder
    CU_DCDR Decoder(
        .br_eq      (beq),
        .br_lt      (blt),
        .br_ltu     (bltu),
        .opcode     (IF_ID.instr[6:0]),
        .func7      (IF_ID.instr[30]),
        .func3      (IF_ID.instr[14:12]),
        .int_taken  (1'b0),
        .ALU_FUN    (alu_ctrl),
        .PC_SEL     (pc_ctrl),
        .srcA_SEL   (alu_a_ctrl),

```

```

        .srcB_SEL  (alu_b_ctrl),
        .RF_SEL    (rf_wr_ctrl),
        .mem_we     (mem_we),
        .rf_we(rf_we),
        .branch(branch_Happen),
        .jal(jal_Happen),
        .jalr(jalr_Happen)

    ); // RegWrite from the write back stage is supposed to go here?
wire[4:0] RS1D =IF_ID.instr[19:15];
wire[4:0] RS2D = IF_ID.instr[24:20];

// ID?EX register
always_ff @(posedge CLK) begin
if (RESET || FlushE) begin
    ID_EX <= 0;
end else begin
    ID_EX.instr      <= IF_ID.instr;
    ID_EX.rs1        <= rs1;
    ID_EX.rs2        <= rs2;
    ID_EX.U_Type     <= U_Type;
    ID_EX.I_Type     <= I_Type;
    ID_EX.S_Type     <= S_Type;
    ID_EX.B_Type     <= B_Type;
    ID_EX.J_Type     <= J_Type;
    ID_EX.rf_we       <= rf_we;
    ID_EX.rf_wr_ctrl  <= rf_wr_ctrl;
    ID_EX.rf_wa       <= rf_wa;
    ID_EX.mem_we      <= mem_we;
    ID_EX.alu_a_ctrl  <= alu_a_ctrl;
    ID_EX.alu_b_ctrl  <= alu_b_ctrl;
    ID_EX.alu_ctrl    <= alu_ctrl;
    ID_EX.pc_ctrl     <= pc_ctrl;
    ID_EX.pc          <= IF_ID.pc;
    ID_EX.pc_4        <= IF_ID.pc_4;
    // fowardind
    ID_EX.RS1         <= RS1D;
    ID_EX.RS2         <= RS2D;
    ID_EX.branch_Happen <=branch_Happen;
    ID_EX.jal_Happen   <=jal_Happen;
    ID_EX.jalr_Happen  <=jalr_Happen;
end
end

//==== Execute
=====

```

```

// Hazard Mux
four_input_Mux #(n(32)) HazardMuxA (
    .SEL    (ForwardMuxA),
    .D0     (ID_EX.rs1),
    .D1     (regdata),
    .D2     (EX_MEM.alu_res),
    .D3     (32'h0000_0000), // dont need this input
    .D_OUT  (HazardMuxA_out) );

four_input_Mux #(n(32)) ALUMUXA (
    .SEL    (ID_EX.alu_a_ctrl),
    .D0     (HazardMuxA_out),
    .D1     (ID_EX.U_Type),
    .D2     (32'd0),
    .D3     (32'd0),
    .D_OUT  (alu_a_mux) );

//Hazard Mux
four_input_Mux #(n(32)) HazardMuxB (
    .SEL    (ForwardMuxB),
    .D0     (ID_EX.rs2),
    .D1     (regdata),
    .D2     (EX_MEM.alu_res),
    .D3     (32'h0000_0000), // dont need this input
    .D_OUT  (HazardMuxB_out) );

four_input_Mux #(n(32)) ALUMUXB (
    .SEL    (ID_EX.alu_b_ctrl),
    .D0     (HazardMuxB_out),
    .D1     (ID_EX.I_Type),
    .D2     (ID_EX.S_Type),
    .D3     (ID_EX.pc),
    .D_OUT  (alu_b_mux) );

//THE ALU
ALU ALU(
    .srcA    (alu_a_mux),
    .srcB    (alu_b_mux),
    .result  (alu_res),
    .alu_fun (ID_EX.alu_ctrl) );

// Branching Thingsss
BranchCondGen branch_logic (
    .rs1     (HazardMuxA_out),
    .rs2     (HazardMuxB_out),
    .br_eq   (beq),

```

```

        .br_lt    (blt),
        .br_ltu   (bltu)
    );

    BranchAddrGen branch_generator (
        .J_Type    (ID_EX.J_Type),
        .B_Type    (ID_EX.B_Type),
        .I_Type    (ID_EX.I_Type),
        .rs1       (ID_EX.rs1),
        .PC        (ID_EX.pc),
        .branch     (branch),
        .jal       (jal),
        .jalr      (jalr)
    );

    assign is_branchE = ID_EX.branch_Happen;
    assign is_jalE   = ID_EX.jal_Happen;
    assign is_jalrE  = ID_EX.jalr_Happen;

    assign branch_cond = (ID_EX.instr[14:13]==2'b00 ? beq ^ ID_EX.instr[12] :
                          ID_EX.instr[14:13]==2'b10 ? blt ^ ID_EX.instr[12] :
                          bltu ^ ID_EX.instr[12]);

    assign branch_takenE = is_branchE && branch_cond;

    assign actual_pc_selE = branch_takenE ? 3'd2 : // B-target
                               is_jalrE   ? 3'd1 : // JALR-target
                               is_jalE    ? 3'd3 : // JAL-target
                               3'd0; // PC+4

    // EX?MEM register
    always_ff @(posedge CLK) begin
        if (RESET)begin
            EX_MEM <= 0;
        end else begin
            EX_MEM.instr      <= ID_EX.instr;
            EX_MEM.alu_res    <= alu_res;
            EX_MEM.rs2       <= HazardMuxB_out;
            EX_MEM.mem_we     <= ID_EX.mem_we;
            EX_MEM.rf_we     <= ID_EX.rf_we;
            EX_MEM.rf_wr_ctrl <= ID_EX.rf_wr_ctrl;
            EX_MEM.rf_wa     <= ID_EX.rf_wa;
            EX_MEM.pc_4      <= ID_EX.pc_4;
        end
    end

    //==== Memory =====

    // assign IOBUS_ADDR = ex_mem_aluRes;
    // assign IOBUS_OUT  = ex_mem_rs2;

```

```

D_MEM DataMem(// reading from instruction mem
               .clk(CLK),
               .dmem_a(EX_MEM.alu_res), // calculated address by ALU
               .dmem_d(EX_MEM.rs2), // the data to put in
               .we(EX_MEM.mem_we),
               .dmem_out(mem_data) );

// MEM?WB register
always_ff @(posedge CLK) begin
  if (RESET)begin
    MEM_WB <= 0;
  end else begin
    MEM_WB.instr      <= EX_MEM.instr;
    MEM_WB.mem_data   <= mem_data;
    MEM_WB.alu_res    <= EX_MEM.alu_res;
    MEM_WB.rf_we      <= EX_MEM.rf_we;
    MEM_WB.rf_wr_ctrl <= EX_MEM.rf_wr_ctrl;
    MEM_WB.rf_wa      <= EX_MEM.rf_wa;
    MEM_WB.pc_4       <= EX_MEM.pc_4;
  end
end

//==== Write Back =====

four_input_mux #(.n(32)) RegMux (
  .SEL  (MEM_WB.rf_wr_ctrl),
  .D0   (MEM_WB.pc_4),
  .D1   (32'h0000_0000),
  .D2   (MEM_WB.mem_data),
  .D3   (MEM_WB.alu_res),
  .D_OUT (regdata) );

//====Hazard Unit code =====

HazardUnit Hazard_Unit(
  .ALUresult_memory(EX_MEM.alu_res),
  .RegWrite_memory(EX_MEM.rf_we),
  .Rd_memory(EX_MEM.rf_wa),
  .Rd_WB(MEM_WB.rf_wa),
  .RegWrite_WB(MEM_WB.rf_we),
  .Rd_execute(ID_EX.rf_wa),
  .Rs1_execute(ID_EX.RS1),

```



```

        .Rs2_execute(ID_EX.RS2),
        .PcSrc_execute(branch_takenE || is_jalE || is_jalrE),
        .Rs1_decode(RS1D),
        .Rs2_decode(RS2D),
        .ResultSrc_execute(ID_EX.rf_wr_ctrl),
        .FowardMuxA_execute(ForwardMuxA),
        .FowardMuxB_execute(ForwardMuxB),
        .Stall_PC(StallF),
        .Stall_IF_ID(StallD),
        .Flush_ID_EX(FlushE),
        .Flush_IF_ID(FlushD)
    );

endmodule

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 05/08/2025 10:29:41 AM
// Design Name:
// Module Name: HazardUnit
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

module HazardUnit(
    input [31:0] ALUresult_memory,
    input RegWrite_memory,
    input [4:0] Rd_memory,
    input [4:0] Rd_WB,
    input RegWrite_WB,
    input [4:0] Rd_execute,
    input [4:0] Rs1_execute,
    input [4:0] Rs2_execute,
    input [4:0] Rs1_decode,
    input [4:0] Rs2_decode,
    input [1:0]ResultSrc_execute,
    input PcSrc_execute,
    output logic [1:0]FowardMuxA_execute,

```

```

output logic [1:0]FowardMuxB_execute,
output logic Stall_PC,
output logic Stall_IF_ID,
output logic Flush_ID_EX,
output logic Flush_IF_ID

);
logic lwStall;

always_comb begin
// initialize variables
    FowardMuxA_execute = 2'b00;
    FowardMuxB_execute = 2'b00;
    Stall_PC          = 1'b0;
    Stall_IF_ID       = 1'b0;
    Flush_ID_EX       = 1'b0;
    lwStall           = 1'b0;
// For Hazard Mux A
    if ((Rs1_execute == Rd_memory) && RegWrite_memory && (Rs1_execute != 0))begin
        FowardMuxA_execute = 2'b10;
    end
    else if ((Rs1_execute == Rd_WB)&& RegWrite_WB && (Rs1_execute !=0)) begin
        FowardMuxA_execute = 2'b01;
    end
    else begin FowardMuxA_execute = 2'b00; end

// For Hazard Mux B
    if ((Rs2_execute == Rd_memory) && RegWrite_memory && (Rs2_execute != 0)) begin
        FowardMuxB_execute = 2'b10;
    end
    else if ((Rs2_execute == Rd_WB)&& RegWrite_WB && (Rs2_execute !=0)) begin
        FowardMuxB_execute = 2'b01;
    end
    else begin FowardMuxB_execute = 2'b00; end

// Stall and flush Logic
    if (((Rs1_decode == Rd_execute)|| (Rs2_decode == Rd_execute)) && (ResultSrc_execute ==
2'b10))
        begin
            lwStall = 1'b1;
        end

    Stall_PC = lwStall;
    Stall_IF_ID = lwStall;
    Flush_ID_EX = lwStall || PcSrc_execute;

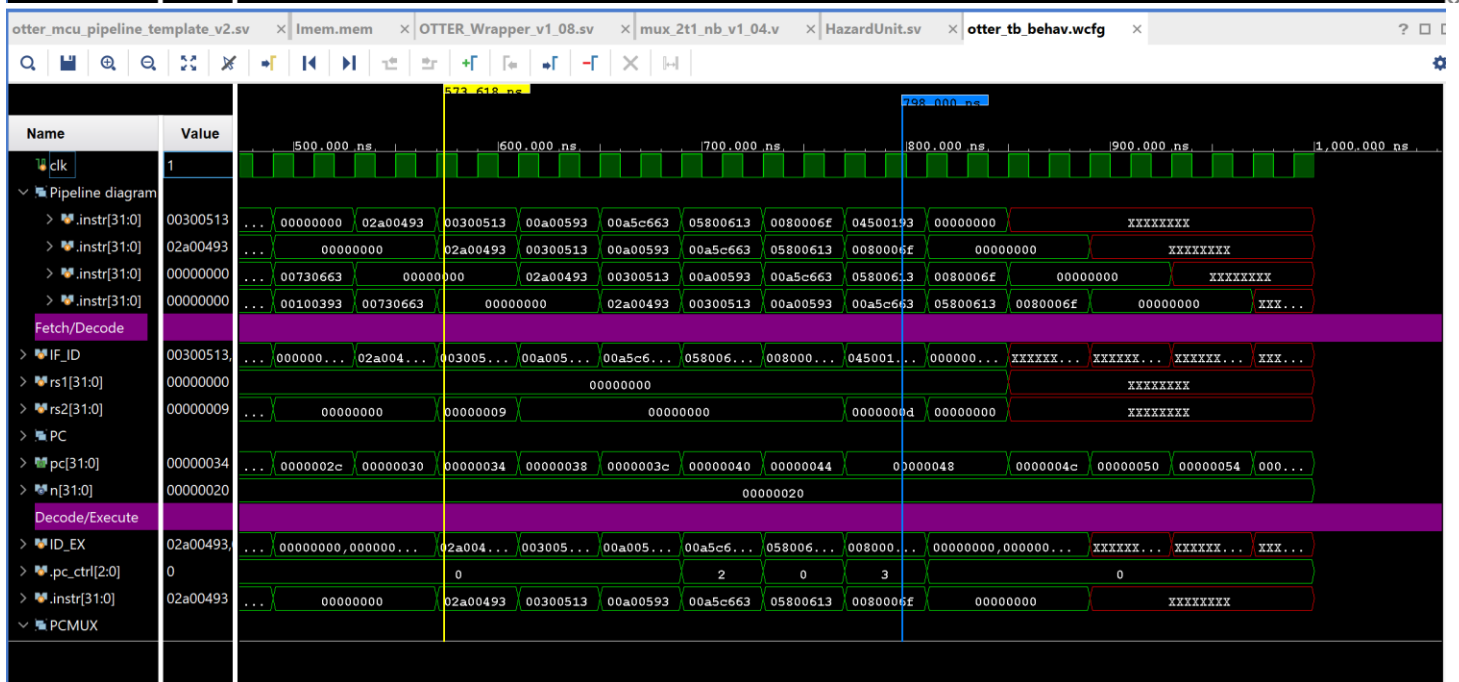
    Flush_IF_ID =PcSrc_execute;
end
endmodule

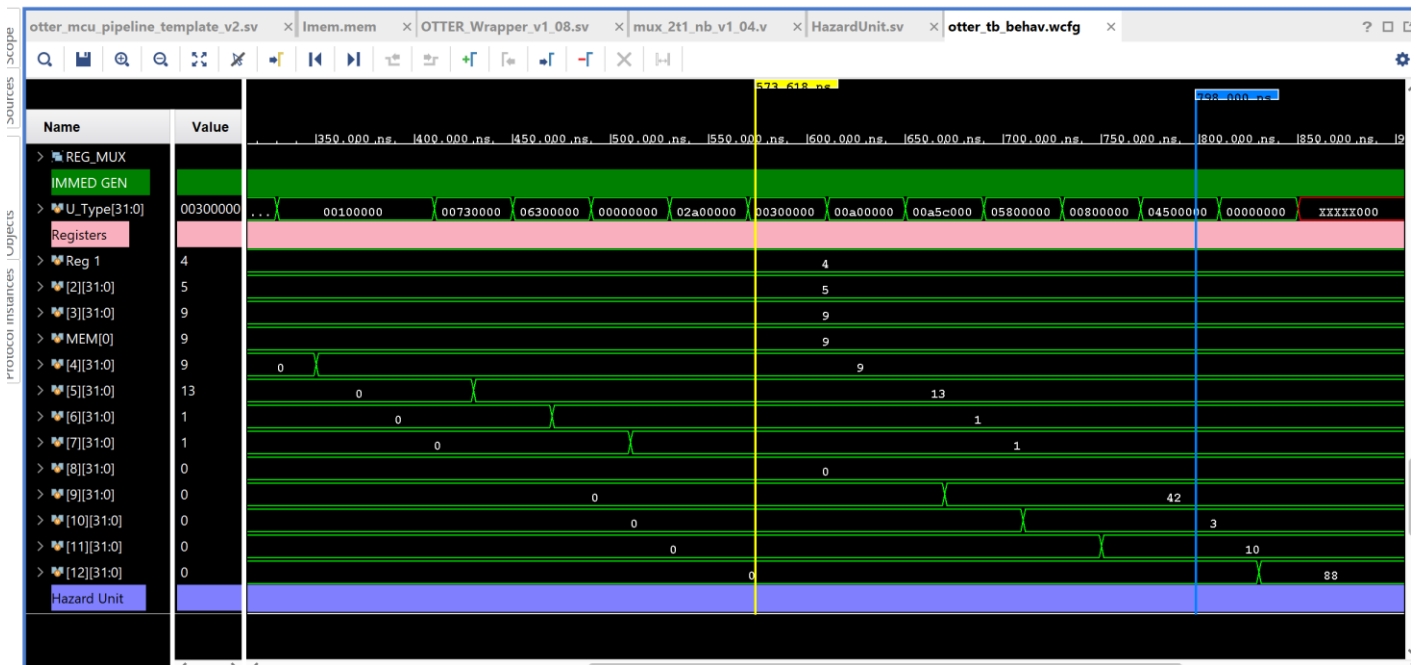
```

Hazard test code Assembly:

```
1  # ALU → ALU forwarding
2  addi x1, x0, 4      # x1 = 4
3  addi x2, x0, 5      # x2 = 5
4  add  x3, x1, x2      # x3 = 9
5
6
7  # Store and then load from memory
8  sw   x3, 0(x0)      # MEM[0] = 9
9  lw   x4, 0(x0)      # x4 = 9 (
10 add  x5, x4, x1      # x5 = 9 + 4 = 13 -> load-use hazard
11
12 # Control hazard test
13 addi x6, x0, 1      # x6 = 1
14 addi x7, x0, 1      # x7 = 1
15 beq  x6, x7, skip   # branch should be taken -> flush next
16 addi x8, x0, 99     # should be flushed so x8 = 0
17 addi x20, x0, 66
18
19 skip:
20 addi x9, x0, 42      # should execute
21
22
23 addi x10, x0, 3      # x10 = 3
24 addi x11, x0, 10     # x11 = 10
25 bgt  x10, x11, Control # branch should not be taken
26 addi x12, x0, 88     # will not be flushed
27 j   Done
28
29 Control:
30 addi x3, x0, 69      # should not run :000
31
32 Done:
```

Verification – Photo proof :





Checking Matrix multiplication :

Summary of Project Design:

First, we started by implementing a solution to RAW Data Hazards . This was done by adding two muxes in the Execute stage right before the ALU muxes.

The first Mux (named HazardMuxA) holds the values, rs1 from the ID/EX register, the write data for the RegFile, and the ALU result from the Mem/WB register. The second mux (named HazardMuxB) holds all the same values except it holds rs2 instead of rs1 . Both muxes have selects that are driven by a hazard unit. The selects are very similar to one another except HazardMuxA is dependent on rs1 while HazardMuxB is dependent on rs2.

First, we are checking to see if a RAW hazard is occurring by checking if the source register is the same as the destination register and then we check if the Regfile write enable is on, to see if the instruction is writing to a register. If this does occur, we set HazardMuxA's output to the output of the ALU held in the EX/MEM register so that value is getting forwarded to the rs1 input of ALU Mux A and the calculation can properly occur. If this did not happen rs1 wouldn't have been written in time for the next instruction to read the correct value and execute the instruction resulting in an incorrect "old" value being used .

The second check is fairly like the first one expect we are checking for values that have already reached the writeback stage and comparing to rs1 trying to catch RAW hazards aswell . The reason we do this is if there's a gap between the two instructions that participate in the RAW hazard or if there is a one cycle stall . This second check allows for forwarding to occur even if the previous check barely missed the values it needed to compare by one cycle. If that miss were to occur the appropriate forwarding would not have happened , and you could have incorrect values being computed or assigned messing up the intention of the code.

For the next two checks we repeated the two checks we did for rs1 expect now for rs2. To add on, if all those checks were to hold false then the hazard mux defaults to original inputs of rs1 and rs2 of the Regfile and no forwarding is needed.

Our fifth check is to catch load-use hazards and whether we need to enable Stalls or Flushes in our pipeline. This is determined by checking again if the source register and the destination register are the same, but we have an additional check to see if a load instruction is also occurring. If all these things occur, then we enable a Stall in the PC and IF/ID register and Flush the ID/EX register.

The final check also asserts the Flush of the ID/Ex register if a branch was actually taken, if there was a jump or a jump and link. This check is added because our Otter implements a branch not taken predictor, so if a branch is taken the additional instruction that was fetched needs to be flushed and the correct instruction needs to be loaded in order for the program to run correctly.

One of the biggest bugs that occurred was implementing the branch not taken predictor and the flushes that came with a correct prediction or misprediction. The flushing and implementation of the correct instructions were not occurring for a good while, but it ended up being cause of incorrect implementation. What happened is that the select of the PC was being driven from the decoder which would pick a select based from what type of instruction, but the branch condition generator was computing whether a branch was taken or not in the execute stage of the pipeline. This was causing all the instructions to just flow through and ignore any branches therefore not needing flushes. After that was realized we moved the computation of the PC select to the execute stage and just had the decoder drive, branch, jal and jalr flags that were pushed to execute. We then wrote some combinational logic that would compute the correct PC_select based on if a branch happened, what type of branch happened, and if the requirements of that branch were met (is the branch taken or not) which fixed our issues and correctly drove flushes and stalls.

Other than that other issues mainly consisted of pulling values from incorrect registers and defining wires incorrectly resulting in wires in the High impedance state or being disconnected. All those wires were fixed by correctly formatting the code and checking various pipeline registers to see where values were being lost.

Performance Benchmark Comparison:

Comparison of pipelined otter with data collected from lab0 on the following:

1. Speedup

Using the 50x50 matrix:

$$X = \text{performance of Lab 2} = 1 / \text{Execution time} = 1 / 1,204,450,410.000 \text{ ns}$$

$$Y = \text{performance of Lab 4} = 1 / \text{Execution time} = 1 / 1200 \text{ ns}$$

$$\text{Speedup} = \text{performance}(x) / \text{performance}(y) = 1,204,450,410 / 1,200 = 1,003,708.675$$

2. Matrix benchmark programs completion time

For lab 2: 1,204,450,410.000 ns

For lab 4: 1200 ns

3. Area and power

The pipelined otter's for Lab 4: dynamic power = 0.03W being 32% of the power, and static power = 0.063W, which is 68% of the power. Total on-chip power consumption is 0.093 W (93 mW).

Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
OTTER_Wrapper	595	582	64	30	2
RISCV_MCU (OTTER_MCU)	594	553	64	0	0
ALU (ALU)	24	0	0	0	0
branch_generator (BranchGenerator)	11	0	0	0	0
branch_logic (BranchControl)	10	0	0	0	0
DataMem (D_MEM)	437	0	64	0	0
PC_Counter (PC_Counter)	24	31	0	0	0
RegFile (RegFile)	49	0	0	0	0

Tcl Console
Messages
Log
Reports
Design Runs
Power
Utilization

Summary

Settings
Summary (0.093 W, Margin: N/A)
Power Supply
Utilization Details
Hierarchical (0.03 W)
Clocks (0.001 W)
Signals (0.011 W)
Data (0.011 W)
Clock Enable (<0.001 W)
Set/Reset (<0.001 W)
Logic (0.007 W)
BRAM (0.011 W)
I/O (<0.001 W)

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: 0.093 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.5°C
Thermal Margin: 74.5°C (14.8 W)
Ambient Temperature: 25.0 °C
Effective θ_{JA} : 5.0°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

Category	Power (W)	Percentage (%)
Dynamic	0.030 W	32%
Device Static	0.063 W	68%
Clocks	0.001 W	2%
Signals	0.011 W	37%
Logic	0.007 W	25%
BRAM	0.011 W	36%
I/O	<0.001 W	0%

Lab 2's design the area can be summarized as:

Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
OTTER_Wrapper	595	582	64	30	2
RISCV_MCU (OTTER_MCU)	594	553	64	0	0
ALU (ALU)	24	0	0	0	0
branch_generator (BranchGenerator)	11	0	0	0	0
branch_logic (BranchCondition)	10	0	0	0	0
DataMem (D_MEM)	437	0	64	0	0
PC_Counter (PC_Counter)	24	31	0	0	0
RegFile (RegFile)	49	0	0	0	0

LUT's used: $1318/20,800 = 6.34\%$

FF's: $160/41,600 = 0.38\%$

BRAMS: $15/50 = 32\%$

I/O's: $46/106 = 43.4\%$

This is Lab2's power:

